

EVA-2: Evaluación y Pauta de Trabajo BACKEND

Asignatura: Desarrollo Backend

Unidad de Competencia: Aplicación web de proceso de formularios

Ponderación Total de la Asignatura: 25%

Puntaje Total del Instrumento: 100 Puntos

Exigencia: 60% para Nota 4.0

I. Instrucciones de la Evaluación (Entregable para el Estudiante)

1. Presentación y Requerimientos Generales

El objetivo de esta evaluación es medir la capacidad técnica en la construcción de servicios Backend con **Django REST Framework (DRF)** y **PostgreSQL**, integrando autenticación JWT basada en roles, persistencia de carro de compras, control transaccional de inventario, filtrado y documentación de API.

- **Modalidad:** Elaboración individual.
- **Base de Datos:** Configuración nativa con **PostgreSQL o MySQL**(no SQLite).
- **Nombre del Proyecto:** El proyecto Django debe ser nombrado adecuadamente en relación con la temática del sistema asignado.
- **Documentación del Código:** Todo el código fuente (modelos, vistas, serializadores, señales) debe estar **comentado en bloques** indicando la lógica implementada.
- **Pie de Página (Footer):** Para aquellos endpoints o vistas HTML base utilizadas, se deben mantener visibles los datos del alumno: **Nombre Completo, Sección y Año**.
- **Soporte de Documentación API:** Debe integrarse **Swagger / OpenAPI** (ej. mediante drf-spectacular o drf-yasg según corresponda).

2. Especificaciones Técnicas del Desarrollo (30 Puntos / 30%)

A. Autenticación y Autorización (JWT)

- Implementación de tokens access y refresh.
- Payload personalizado (claims) que incluya el **Rol** del usuario (Cliente vs Administrador/Gestor).
- Aplicación de permisos en DRF: Endpoints públicos para lectura de catálogo, protegidos (IsAuthenticated) para clientes y restringidos (IsAdminUser o permisos personalizados) para gestión de inventario y cambio de estados.

B. Modelo de Datos y PostgreSQL

- Configuración del motor `django.db.backends.postgresql`.
- Implementación explícita de al menos una propiedad CHOICES en el modelo (ej. estados de orden/transacción).
- Múltiples modelos relacionados con integridad referencial (Foreign Keys, OneToOne).

C. Carro de Compras Persistente

- Relación 1 a 1 entre el Usuario y su Carro activo en base de datos.
- Los productos/ítems agregados **deben persistir en PostgreSQL** aun cuando el usuario cierre sesión (logout) o cambie de dispositivo.

D. Transacciones, Stock y Estados

- **Checkout:** Al confirmar el carro, se genera el registro histórico de la Orden.
- **Descuento de Inventario:** El stock o cupo **NO** se descuenta al agregar al carro, sino en el momento exacto en que la transacción se valida como PAGADO.
- **Manejo de Stock:** Si el stock es insuficiente al pagar, la transacción se rechaza. Si la orden se actualiza a CANCELADO, el stock debe reponerse automáticamente al catálogo.

E. Filtros y Búsqueda

- Sistema de filtros aplicado sobre campos clave del modelo mediante django-filter (ej. filtrado por categoría, rango de precios, etc.).

3. Defensa Oral y Preguntas Técnicas (70 Puntos / 70%)

Cada estudiante presentará y **defenderá de manera individual su código e implementación** ante el docente. Se realizarán preguntas de validación teórica y práctica sobre la arquitectura, el flujo de datos, la seguridad JWT y las consultas a base de datos para certificar la autoría y dominio del sistema desarrollado.

II. Pauta de Cotejo Técnica (Checklist de Entrega)

Elemento	Requerimiento Técnico	Estado (✓)
Base de Datos	Conexión activa a PostgreSQL en settings.py	[]
Documentación	Swagger / OpenAPI operativo en /api/docs/	[]
Comentarios	Código documentado en bloques explícitos	[]
Datos Alumno	Nombre, Sección y Año presentes en la vista/footer base	[]
Modelos	Atributo con CHOICES definido	[]

Elemento	Requerimiento Técnico	Estado (✓)
Filtros	django-filter configurado en endpoints de consulta	[]
Autenticación	Login JWT retornando tokens y claims de rol	[]
Carro	Persistencia post-logout en PostgreSQL	[]
Stock/Cupos	Validación y descuento atómico al cambiar a estado PAGADO	[]

III. Rúbrica Analítica de Evaluación (100 Puntos Totales)

Criterio 1: Desarrollo Técnico y Lógica de Negocio Backend (30% - 30 Puntos)

Indicador	Sobresaliente (100%)	Logrado (70%)	En Desarrollo (40%)	No Logrado (0%)	Puntaje
Conexión DB, CHOICES y Filtros	Configura PostgreSQL correctamente. Implementa CHOICES en los modelos y sistema de django-filter en los endpoints requeridos. (6 Pts)	Configura PostgreSQL y CHOICES, pero los filtros presentan fallos menores de búsqueda. (4.2 Pts)	Usa PostgreSQL pero no implementa CHOICES o los filtros no están operativos. (2.4 Pts)	No configura PostgreSQL (usa SQLite) ni implementa filtros o CHOICES. (0 Pts)	/ 6
Autenticación JWT y Roles	Autenticación JWT funcional con claims de rol en los tokens. Permisos DRF correctamente aplicados según	Implementa JWT funcional pero no incluye claims de rol en el token, o la restricción de	Genera tokens JWT básicos pero no aplica restricción de permisos por rol en	No implementa JWT o la autenticación no funciona. (0 Pts)	/ 8

Indicador	Sobresaliente (100%)	Logrado (70%)	En Desarrollo (40%)	No Logrado (0%)	Puntaje
	endpoints. (8 Pts)	endpoints es incompleta. (5.6 Pts)	las vistas. (3.2 Pts)		
Persistencia del Carro	Carro de compras persistente en DB vinculado al usuario. Mantiene los ítems tras cerrar sesión y en reconexión. (8 Pts)	Carro vinculado a DB, pero pierde ítems bajo ciertos estados o no valida duplicados de productos. (5.6 Pts)	El carro depende de la sesión temporal o frontend y no persiste en DB PostgreSQL. (3.2 Pts)	No implementa carro de compras ni estructura de ítems. (0 Pts)	/ 8
Lógica de Stock y Transacciones	Transición de estados de orden correcta. El stock se descuenta únicamente al pasar a PAGADO y se reestablece si se CANCELA. (8 Pts)	Se realiza el checkout y cambio de estados, pero descuenta stock al agregar al carro o no lo repone al cancelar. (5.6 Pts)	Genera la orden pero no realiza ningún tipo de control o descuento sobre el stock del catálogo. (3.2 Pts)	No gestiona órdenes, estados ni lógica de inventario. (0 Pts)	/ 8

Criterio 2: Defensa Individual y Preguntas Técnicas (70% - 70 Puntos)

Evaluación oral individual sobre el código desarrollado.

Indicador	Sobresaliente (100%)	Logrado (70%)	En Desarrollo (40%)	No Logrado (0%)	Puntaje
1. Dominio de la Arquitectura, Configuración DB y Modelos	Explica con precisión la configuración de PostgreSQL en Django, la estructura de las tablas, relaciones (1:1, 1:N) y el uso explícito de CHOICES. (12 Pts)	Explica la mayoría de los modelos y la conexión a DB, pero presenta dudas menores al justificar relaciones complejas o campos específicos. (8.4 Pts)	Muestra un conocimiento superficial de la base de datos; no logra justificar las relaciones ni la configuración del motor. (4.8 Pts)	No logra explicar la estructura del proyecto ni la configuración de la base de datos. (0 Pts)	/ 12
2. Defensa del Flujo JWT, Claims de Rol y Permisos RBAC	Explica detalladamente la generación de tokens JWT, la inclusión de claims del rol (Espectador/Organizador) y la restricción de endpoints mediante permisos de DRF. (12 Pts)	Explica el flujo de autenticación JWT, pero vacila sobre cómo se leen los claims de rol o cómo interactúan con las clases de permiso. (8.4 Pts)	Reconoce la librería JWT, pero no comprende el funcionamiento de los tokens ni cómo se aplican los permisos en los endpoints. (4.8 Pts)	No responde preguntas sobre autenticación JWT ni sobre la seguridad de las vistas. (0 Pts)	/ 12
3. Defensa del Carro de Compras Persistente	Justifica técnicamente la persistencia del carro en PostgreSQL asociada al ID del usuario, demostrando cómo se conservan los ítems tras un logout o reconexión. (12 Pts)	Explica el funcionamiento del carro en la base de datos, pero no ubica o duda al mostrar la consulta/relación que garantiza la persistencia. (8.4 Pts)	Describe el carro de forma teórica, pero su implementación depende de sesiones temporales o no persiste en PostgreSQL. (4.8 Pts)	No logra explicar la lógica de la entidad carro ni su persistencia en la base de datos. (0 Pts)	/ 12
4. Defensa del Ciclo Transaccional,	Explica con exactitud el flujo del checkout, la generación de la orden, el descuento atómico de	Explica la transacción y el cambio de estados de la	Describe conceptualmente la compra, pero no	No explica la gestión transaccional, el manejo de	/ 12

Indicador	Sobresaliente (100%)	Logrado (70%)	En Desarrollo (40%)	No Logrado (0%)	Puntaje
Checkout y Stock	stock al pasar a PAGADO y la emisión de entradas únicas (UUID). (12 Pts)	orden, pero presenta dudas en el bloque de código que valida o reestablece el stock. (8.4 Pts)	identifica la lógica en sus views/serializers que descuenta el stock o genera los tickets. (4.8 Pts)	stock ni la actualización de estados de la orden. (0 Pts)	
5. Explicación de Filtros, Búsqueda y Documentación OpenAPI	Demuestra el funcionamiento e integración de django-filter en los endpoints de consulta y la generación automática de la documentación Swagger/OpenAPI. (10 Pts)	Explica la documentación de la API, pero presenta inconsistencias al justificar la configuración o sintaxis de los filtros aplicados. (7 Pts)	Identifica la ruta de Swagger, pero no logra explicar cómo se configuraron los filtros ni sobre qué campos operan. (4 Pts)	No explica el sistema de filtros ni la documentación de endpoints. (0 Pts)	/ 10
6. Justificación de Calidad del Código, Comentarios y Footer	Justifica la legibilidad de su código, los comentarios en bloques y la renderización de los datos del alumno (nombre, sección, año) en el footer base. (12 Pts)	Muestra los comentarios y los datos en el footer, pero carece de claridad al explicar la estructura del código o decisiones de refactorización. (8.4 Pts)	Cumple parcialmente con la entrega de datos, pero el código no está comentado en bloques o no puede defender las decisiones de diseño. (4.8 Pts)	El código no contiene comentarios, no incluye datos en el footer ni puede justificar la autoría del desarrollo. (0 Pts)	/ 12

Requerimientos Transversales de Arquitectura

Todos los proyectos deben cumplir obligatoriamente con la siguiente línea base técnica:

1. Autenticación & Autorización JWT:

- Utilizar tokens JWT (access y refresh).
- Los claims del token deben incluir el **Rol** del usuario (Cliente/Usuario vs Administrador/Gestor).
- **Permisos DRF:** Endpoints de lectura pública para catálogo/oferta; endpoints protegidos con `IsAuthenticated` para la gestión del carro y transacciones del cliente; endpoints protegidos con `IsAdminUser` o permisos personalizados por rol para gestión de inventario y cambio de estados.

2. Persistencia del Carro de Compras (Post-Logout):

- El "carro" debe ser un recurso persistente en la Base de Datos vinculado al ID del usuario autenticado (Relación 1 a 1 entre Usuario y Carro activo).
- Al cerrar sesión o iniciar sesión desde otro dispositivo, los ítems agregados previamente deben conservarse intactos.

3. Ciclo de Vida de la Transacción y Gestión de Inventario:

- **Estado del Carro:** *Activo* (mientras se agregan/quitan ítems).
- **Transición a Orden:** Al ejecutar el endpoint de checkout, el contenido del carro se liquida y se genera un registro histórico de Orden/Transacción.
- **Estados de la Transacción:** PENDIENTE -> PAGADO -> ENTREGADO/COMPLETADO (o CANCELADO).
- **Descuento de Stock/Cupos:** El inventario o disponibilidad de cupos **NO** se descuenta al añadir al carro, sino en el momento exacto en que la orden pasa a estado PAGADO (o al confirmar la transacción). Si el stock es insuficiente al momento de pagar, la transacción debe ser rechazada. Si la orden es CANCELADA, se debe reponer el stock descontado.

4. Publicación en GitHub:

- Su proyecto debe estar publicado en Github antes de la hora límite de entrega, sino se considera fuera de plazo y se evaluará con nota mínima 1.0.-

Proyectos disponibles para la evaluación.

Proyecto 1: Tienda de Hardware y Componentes PC (Retail)

Descripción Narrativa del Negocio

E-Commerce especializado en componentes informáticos. El sistema maneja usuarios clasificados como **Clientes y Administradores de TI**. Los Administradores gestionan las categorías (Procesadores, Tarjetas de Video, RAM) y los productos. Cada producto posee nombre, marca, precio unitario, código SKU, descripción y un stock físico disponible.

El **Cliente** puede explorar el catálogo, ver el detalle de cada componente y mantener un carro de compras persistente. Dentro del carro, puede agregar componentes, modificar la cantidad solicitada o eliminar ítems. Al procesar la compra (Checkout), el sistema genera una Orden de Compra que congela los precios de los productos en ese instante histórico (evitando que cambios futuros en el precio del catálogo afecten la venta realizada). Al confirmarse el pago, el sistema verifica que haya stock suficiente para cada producto del carro; si se valida, descuenta el stock del catálogo y actualiza la orden al estado PAGADO. Posteriormente, un Administrador puede cambiar el estado a ENTREGADO o CANCELADO.

MATRIZ DE ROLES Y PERMISOS API

- **PÚBLICO:** GET /API/PRODUCTOS/, GET /API/CATEGORIAS/
- **CLIENTE:** GET/POST/DELETE /API/CARRO/, POST /API/ORDENES/CHECKOUT/, GET /API/MIS-ORDENES/
- **ADMINISTRADOR:** POST/PUT/DELETE /API/PRODUCTOS/, PATCH /API/ORDENES/{ID}/ESTADO/

Proyecto 2: Plataforma de Reservas de Cursos y Bootcamps (EdTech)

Descripción Narrativa del Negocio

Sistema de matriculación para una academia tecnológica. Existen dos roles: **Estudiante** y **Coordinador Académico**. El Coordinador administra las Áreas de Conocimiento y los Cursos/Bootcamps disponibles. Cada curso posee título, descripción, costo de matrícula, fecha de inicio, fecha de término y un límite máximo de cupos por cohorte.

El **Estudiante** navega por la oferta académica y puede agregar múltiples cursos o talleres a su "Carro de Matrícula" persistente. El sistema no permite agregar el mismo curso dos veces al mismo carro activo. Durante el checkout, la aplicación valida que cada curso seleccionado disponga de al menos un cupo libre. Si hay disponibilidad, la matrícula cambia a estado PAGADO, se descuenta un cupo disponible por cada curso del catálogo y se generan las inscripciones oficiales del estudiante. Si la orden es CANCELADA por soporte administrativo, el cupo del curso se libera automáticamente para que otro estudiante pueda matricularse.

MATRIZ DE ROLES Y PERMISOS API

- PÚBLICO: GET /API/CURSOS/, GET /API/AREAS/
- ESTUDIANTE: GET/POST/DELETE /API/CARRO-MATRICULA/, POST /API/MATRICULAS/CONFIRMAR/, GET /API/MIS-MATRICULAS/
- COORDINADOR: POST/PUT/DELETE /API/CURSOS/, PATCH /API/MATRICULAS/{ID}/ESTADO/

Proyecto 3: Venta de Entradas para Eventos y Conciertos (Entretenimiento)

Descripción Narrativa del Negocio

Plataforma de boletaje para eventos masivos. Se definen dos roles mediante JWT: **Espectador** y **Organizador de Eventos**. Los Organizadores registran Recintos y Eventos. Cada evento contiene nombre, artista/banda, fecha/hora de realización y múltiples Sectores/Localidades (ej. *Cancha General*, *VIP*, *Tribuna*), cada uno con su respectivo precio y cantidad de entradas en stock total.

El **Espectador** puede armar una "Reserva Temporal de Entradas" (su carro de compra persistente) agregando una o más entradas para diferentes sectores o eventos. Al confirmar el pedido y procesar el pago, el sistema valida la disponibilidad de entradas para cada sector seleccionado. Si el pago es exitoso, la transacción pasa a estado PAGADO, se descuenta la cantidad de entradas del stock del sector y el sistema emite automáticamente las Entradas Compradas asignando un código Hash/UUID único por ticket. El Organizador puede consultar las ventas y actualizar órdenes a estado CANCELADO (liberando los tickets) o ENTREGADO (marcando asistencia/ingreso).

MATRIZ DE ROLES Y PERMISOS API

- PÚBLICO: GET /API/EVENTOS/, GET /API/EVENTOS/{ID}/SECTORES/
- ESPECTADOR: GET/POST/DELETE /API/CARRO-TICKETS/, POST /API/COMPRAS/PAGAR/, GET /API/MIS-ENTRADAS/
- ORGANIZADOR: POST/PUT/DELETE /API/EVENTOS/, PATCH /API/COMPRAS/{ID}/ESTADO/

Proyecto 4: Reservas de Pasajes de Buses Interurbanos (Transporte)

Descripción Narrativa del Negocio

Sistema de gestión de viajes e itinerarios de autobuses. El sistema distingue entre **Pasajero** y **Administrador de Flota**. El Administrador de Flota gestiona Ciudades/Terminales, Rutas (Origen-Destino), Buses y Servicios/Itinerarios (que vinculan una ruta, un bus específico, la fecha/hora de salida y los asientos asignados con su respectiva tarifa según el tipo: *Semicama* o *Cama*).

El **Pasajero** busca pasajes indicando origen y destino, selecciona un servicio y agrega al "Carro de Pasajes" los asientos deseados (especificando para cada asiento seleccionado el Nombre completo y RUT/Pasaporte del ocupante). Al momento de confirmar la transacción, la API realiza una verificación atómica: comprueba que ninguno de los asientos en el servicio haya cambiado su estado a ocupado. Al procesarse el pago (PAGADO), se descuenta la disponibilidad de asientos del bus, la transacción se cierra y se emiten los boletos de viaje. Si la compra es CANCELADA antes de la salida del bus, el asiento vuelve a estar disponible para la venta.

MATRIZ DE ROLES Y PERMISOS API

- **PÚBLICO:** GET /API/SERVICIOS/BUSCAR/, GET /API/SERVICIOS/{ID}/ASIENTOS/
- **PASAJERO:** GET/POST/DELETE /API/CARRO-PASAJES/, POST /API/VENTAS/CHECKOUT/, GET /API/MIS-BOLETOS/
- **ADMINISTRADOR DE FLOTA:** POST/PUT/DELETE /API/SERVICIOS/, PATCH /API/VENTAS/{ID}/ESTADO/

Proyecto 5: Pedidos de Insumos Médicos y Farmacia (Salud / B2B)

Descripción Narrativa del Negocio

Plataforma de abastecimiento de fármacos e insumos médicos para clínicas e instituciones de salud. El sistema administra dos roles: **Institución Médica (Cliente)** y **Gestor de Bodega Farmacéutica**. El Gestor de Bodega administra las Categorías de Insumos (Material Quirúrgico, Medicamentos, Equipamiento) y los Insumos en stock, registrando nombre comercial, principio activo, lote, fecha de vencimiento, precio unitario por caja y stock disponible en bodega.

La **Institución Médica** autenticada compone una "Solicitud de Abastecimiento" (Carro de Compras B2B) agregando diferentes insumos y especificando la cantidad de cajas requeridas por cada producto. Al ejecutar el envío y pago de la orden, la API valida el stock físico en bodega. Si hay existencias, la orden cambia a PAGADO, se descuenta el stock de cajas de la bodega y se genera la orden de despacho. Si el Gestor de Bodega actualiza el estado de la orden a ENTREGADO, la transacción concluye; si por algún motivo la orden es CANCELADA (por ejemplo, por falla en la cadena de frío), las existencias se reincorporan al inventario de la bodega.

MATRIZ DE ROLES Y PERMISOS API

- **PÚBLICO:** GET /API/INSUMOS/ (CATÁLOGO PÚBLICO O CON RESTRICCIÓN SEGÚN REQUERIMIENTO)
- **INSTITUCIÓN MÉDICA:** GET/POST/DELETE /API/CARRO-INSUMOS/, POST /API/SOLICITUDES/CONFIRMAR/, GET /API/MIS-SOLICITUDES/
- **GESTOR DE BODEGA:** POST/PUT/DELETE /API/INSUMOS/, PATCH /API/SOLICITUDES/{ID}/ESTADO/

Proyecto 6: Arriendo de Maquinaria de Construcción (Renting / Servicios)

Descripción Narrativa del Negocio

Plataforma de arriendo de maquinaria pesada y herramientas industriales. Maneja dos roles: **Empresa Constructora (Cliente)** y **Ejecutivo de Arriendos (Administrador)**. El Ejecutivo de Arriendos cataloga los Equipos (Excavadoras, Generadores, Andamios, Hormigoneras), definiendo su categoría, tarifa diaria de arriendo, monto de garantía fija requerida y la cantidad de unidades físicas disponibles en flota.

La **Empresa Constructora** agrega maquinarias a su "Carro de Arriendo". Para cada máquina agregada dentro del carro, debe especificar la fecha_inicio y la fecha_fin del periodo de uso deseado. El sistema calcula automáticamente el costo del ítem multiplicando la tarifa diaria por la cantidad de días entre ambas fechas,

adicionando el valor de la garantía. Al presionar "Confirmar Contrato", el sistema valida que las unidades físicas solicitadas estén libres en ese rango de fechas. Al registrarse el pago (PAGADO), se descuenta la unidad de la flota disponible para dicho período y se genera el Contrato de Arriendo. El Ejecutivo puede cambiar el estado a ENTREGADO (al retirar el equipo) o COMPLETADO (al devolver el equipo, lo que reincorpora la máquina al inventario disponible).

MATRIZ DE ROLES Y PERMISOS API

- **PÚBLICO: GET /API/MAQUINARIAS/**
- **EMPRESA CONSTRUCTORA: GET/POST/DELETE /API/CARRO-ARRIENDO/, POST /API/CONTRATOS/CHECKOUT/, GET /API/MIS-CONTRATOS/**
- **EJECUTIVO DE ARRIENDOS: POST/PUT/DELETE /API/MAQUINARIAS/, PATCH /API/CONTRATOS/{ID}/ESTADO/**